FIAP — FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ARTHUR DE SOUZA LOPES
EDUARDO OLIVEIRA CONDE
LUCAS BOSSI CRUZ
PEDRO DANIEL MELO DE MORAES
RICK NASCIMENTO DA COSTA

HEAVEN:
plataforma de gamificação sustentável

São Paulo - SP
2026

ARTHUR DE SOUZA LOPES
EDUARDO OLIVEIRA CONDE
LUCAS BOSSI CRUZ
PEDRO DANIEL MELO DE MORAES
RICK NASCIMENTO DA COSTA

HEAVEN:
plataforma de gamificação sustentável

Documentação técnica da API RESTful em Spring Boot, apresentada à disciplina Domain Driven Design using Java, como entrega do 4º Sprint do Challenge 2026, curso de Análise e Desenvolvimento de Sistemas, turma 1TDSPW, da FIAP. Equipe Heaven.

São Paulo - SP
2026

Heaven — Documentação do Java (Sprint 4)

1. Identificação do projeto

Equipe: Heaven **Solução:** Heaven — Plataforma de Gamificação Sustentável (API RESTful em Spring Boot)

Integrantes

Nome	RM
Arthur de Souza Lopes	RM573023
Eduardo Oliveira Conde	RM573163
Lucas Bossi Cruz	RM572876
Pedro Daniel Melo de Moraes	RM569786
Rick Nascimento da Costa	RM571828

Links

- Repositório Java: https://github.com/PedroDaniel7/heaven-java-api
- Repositório do front: https://github.com/PedroDaniel7/heaven-frontend-2semestre
- API em produção (Render): https://heaven-java-api.onrender.com
- Swagger: https://heaven-java-api.onrender.com/swagger-ui/index.html
- Front em produção (Vercel): https://heaven-frontend-2semestre.vercel.app

2. Objetivo e escopo

O Heaven é uma plataforma de gamificação sustentável. O usuário registra ações ambientais (por exemplo, reciclar ou economizar água), outros usuários validam essas ações por voto, e o sistema converte o resultado em pontos, reputação, sequência (streak) e posição no ranking.

Esta entrega é a API RESTful em Spring Boot que serve o front-end React. Ela substitui a versão de console das sprints anteriores e usa o banco Oracle modelado na disciplina de Banco de Dados.

Dentro do escopo: cadastro e login, perfil, catálogo de tipos de ação, registro e cancelamento de ações, validação anônima por voto, ranking público e administração dos tipos de ação.

Fora do escopo desta sprint: upload de arquivos de evidência (a evidência é texto), recuperação de senha, notificações e perfis de acesso além de usuário comum e administrador por chave.

3. Principais funcionalidades

- **Conta:** cadastro (senha com no mínimo 8 caracteres, guardada como hash), login com token e edição do perfil. Excluir a conta desativa o usuário (`INATIVO`) em vez de apagar os dados.
- **Tipos de ação:** consulta pública do catálogo; o administrador cria, edita e remove tipos por chave (`X-Admin-Key`).
- **Ações:** o usuário registra uma ação com quantidade e evidência, acompanha o status (`PENDENTE` , `APROVADA` , `REJEITADA` , `CANCELADA`) e pode cancelar enquanto não houver votos.
- **Validação anônima:** o validador pede uma ação pelo sorteio e vota `APROVAR` ou `REJEITAR` . A oferta fica reservada por 10 minutos e o voto não mostra o autor da ação.
- **Regras no banco:** triggers fecham a ação com pelo menos 3 votos e 60% de aprovação ponderada, e calculam pontos e reputação (+1 e −2). Há limites diários por nível de confiança e bloqueio de autovalidação.
- **Ranking:** lista pública por pontos, com limite de 1 a 100 posições.
- **Erros padronizados:** todo erro devolve `{status, codigo, mensagem, campos[], dataHora}` , sem expor SQL nem stack trace.

Escopo do CRUD. A API entrega CRUD completo para as entidades que o front usa: usuário (cadastro, consulta, edição e desativação) e tipos de ação (criar, listar, editar e excluir, pela área de administração). A ação do usuário tem criação, consulta e cancelamento; não há edição de ação já registrada. As demais entidades do modelo (nível de confiança, histórico de pontos e de reputação, conquistas, incidentes de fraude, penalizações e parâmetros do sistema) têm classe de modelo e DAO, mas não são consumidas pelo front nesta sprint e por isso não têm endpoint próprio.

4. Arquitetura e tecnologias

Tecnologias: Java 17, Spring Boot 3.5, Maven Wrapper, JDBC puro (sem JPA) com pool Hikari, Oracle (driver ojdbc11), Swagger/OpenAPI, Docker, Render (API) e Vercel (front React).

Camadas e pacotes

Pacote	Responsabilidade
`resource`	Controllers REST: recebem a requisição, validam o DTO e devolvem o status HTTP. Inclui o `ApiExceptionHandler`, que padroniza os erros.
`dto`	Records de entrada e saída. A API nunca devolve hash de senha, peso de voto nem lista de votos.
`bo`	Regras de negócio e controle de transação (`UsuarioBO`, `AcaoBO`, `ValidacaoBO`, `TipoAcaoAdminBO`, todos sobre o `BaseBO`). Para falar com o Oracle, os BOs usam os DAOs JDBC (pacote `br.com.heaven.dao`) e as classes de modelo (`br.com.heaven.model`: `Usuario`, `TipoAcao`, `AcaoUsuario`, `NivelConfianca`).
`service`	Apoio: token (`TokenService`), hash de senha, constantes de regra e exceções de negócio com código de erro.
`config`	Pool de conexões, `CorsFilter`, `AuthInterceptor` (nega por padrão; `@Publico` e `@SomenteAdmin` liberam) e `WebConfig`.

Decisões de projeto

- **Autenticação sem Spring Security:** token HMAC-SHA256 com validade de 12 horas. O `usuarioId` vem sempre do token, nunca do corpo da requisição.
- **Regras críticas no banco:** o fechamento da votação, os pontos e a reputação ficam em triggers do Oracle, para garantir consistência mesmo fora da API.
- **Credenciais só por variável de ambiente**, nunca no código.
- **Exclusão lógica:** conta e ação são desativadas ou canceladas, não apagadas.
- **Sorteio de validação:** a oferta é guardada em memória por 10 minutos; após isso o voto devolve `OFERTA_EXPIRADA`.
- **Conexão com o banco:** a classe `DataSourceConfig` (pacote `config`) cumpre o papel de fábrica de conexões (padrão Connection Factory): cria o pool Hikari a partir das variáveis de ambiente, e os DAOs obtêm as conexões dele.

5. Tabela de endpoints (API RESTful)

Base: `/api`. Acesso “Token” exige `Authorization: Bearer <token>`; “Admin” exige o header `X-Admin-Key`.

Verbo	URI	Acesso	Sucesso	Erros
GET	`/api/saude`	Público	200	—
GET	`/api/saude/banco`	Público	200	503
POST	`/api/usuarios`	Público	201	400, 409
POST	`/api/auth/login`	Público	200	400, 401, 403
GET	`/api/usuarios/me`	Token	200	401
PUT	`/api/usuarios/me`	Token	200	400, 401, 409
DELETE	`/api/usuarios/me`	Token	204	401
GET	`/api/tipos-acao`	Público	200	—
GET	`/api/tipos-acao/{id}`	Público	200	400, 404
POST	`/api/acoes`	Token	201	400, 401, 403, 404, 422
GET	`/api/acoes/minhas`	Token	200	401
GET	`/api/acoes/{id}`	Token	200	400, 401, 404
DELETE	`/api/acoes/{id}`	Token	204	400, 401, 404, 409
POST	`/api/validacoes/sorteio`	Token	200 (204 sem ação elegível)	401, 403, 422
POST	`/api/validacoes`	Token	201	400, 401, 403, 404, 409, 422
GET	`/api/ranking?limite=20`	Público	200	400
GET	`/api/admin/tipos-acao`	Admin	200	401
POST	`/api/admin/tipos-acao`	Admin	201	400, 401, 409
PUT	`/api/admin/tipos-acao/{id}`	Admin	200	400, 401, 404, 409
DELETE	`/api/admin/tipos-acao/{id}`	Admin	204	400, 401, 404, 409

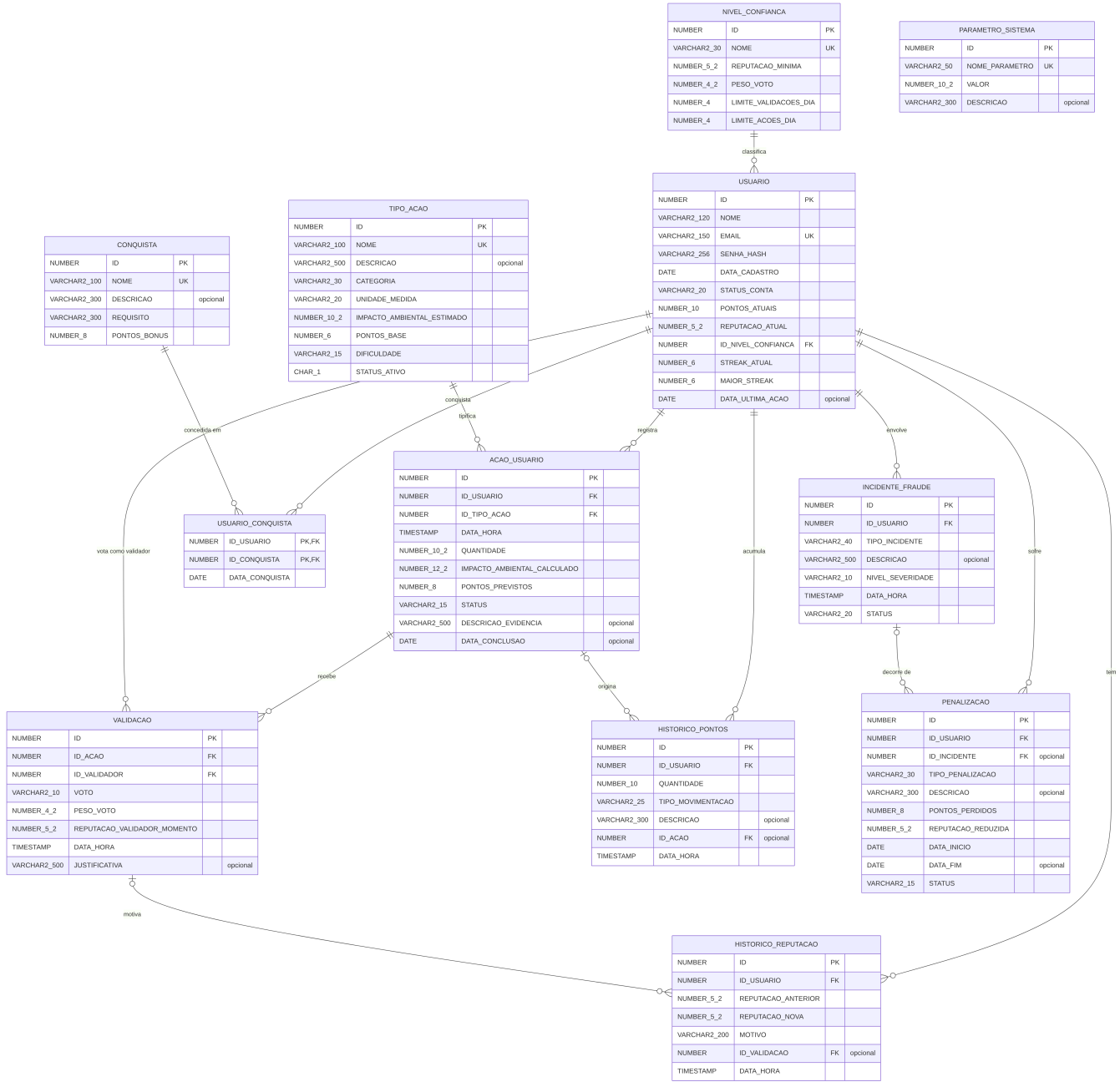
Formato de erro (todos os endpoints): `{status, codigo, mensagem, campos[], dataHora}`, com `codigo` em `VALIDACAO` (400), `NAO_AUTENTICADO` (401), `PROIBIDO` (403), `NAO_ENCONTRADO` (404), `CONFLITO` (409), `OFERTA_EXPIRADA` (409), `REGRA` (422),

`BANCO_INDISPONIVEL` (503) e `ERRO_INTERNO` (500).

Documentação interativa: Swagger em `/swagger-ui/index.html` e especificação em `/v3/api-docs`.

6. Modelo entidade-relacionamento (MER)

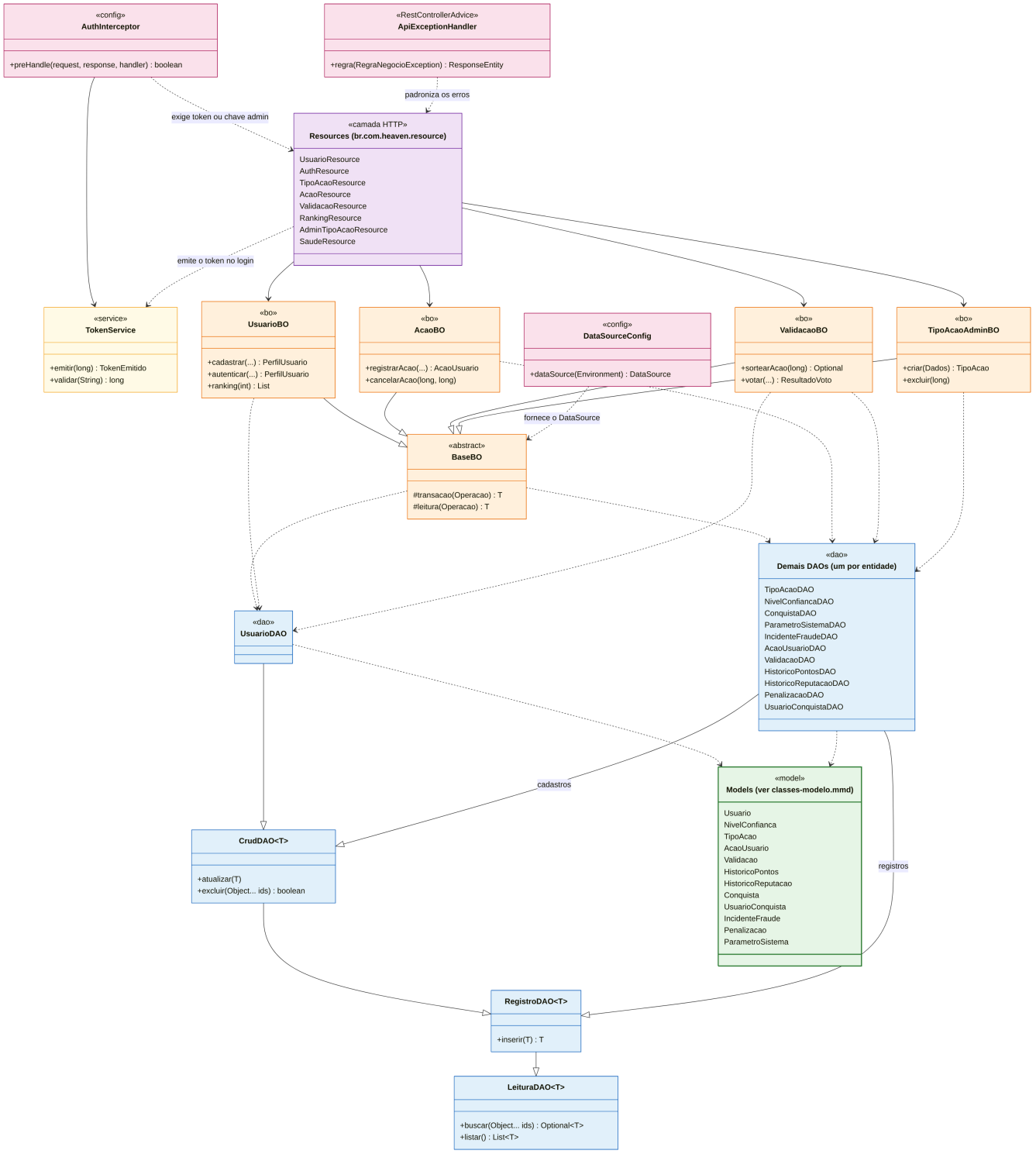
O banco Oracle tem 12 tabelas, 3 views e 5 triggers, criados na disciplina de Banco de Dados. O diagrama abaixo foi gerado a partir do DDL real. As tabelas `ACAO_USUARIO` e `VALIDACAO` são o núcleo da regra de negócio: os triggers fecham a votação e atualizam pontos e reputação.



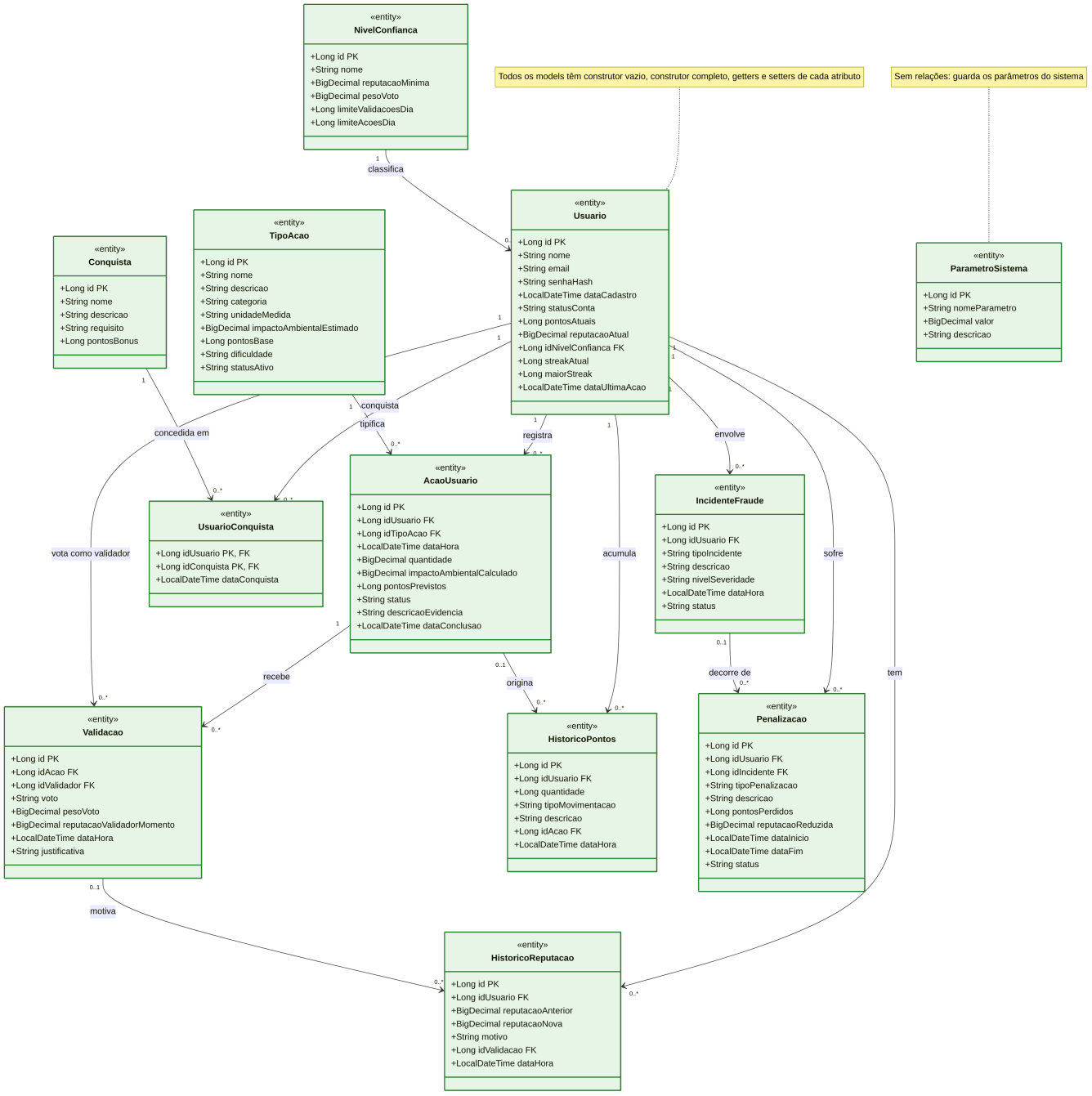
Fonte do diagrama no repositório: `docs/mer.mmd`.

7. Diagrama de classes

O diagrama de classes está dividido em duas figuras, para ficarem legíveis: a visão por camadas (arquitetura) e o modelo de dados (as 12 entidades). Na visão por camadas, os `resource` chamam os BOs, os BOs herdam de `BaseBO`, obtêm a conexão pelo `DataSourceConfig` e acessam o banco pelos DAOs, que mapeiam os modelos. Os DAOs seguem a hierarquia `LeituraDAO` ← `RegistroDAO` ← `CrudDAO`; os demais DAOs aparecem agrupados numa só caixa.



Modelo de dados. As 12 classes de `br.com.heaven.model`, com atributos, chaves e relações iguais às do MER.



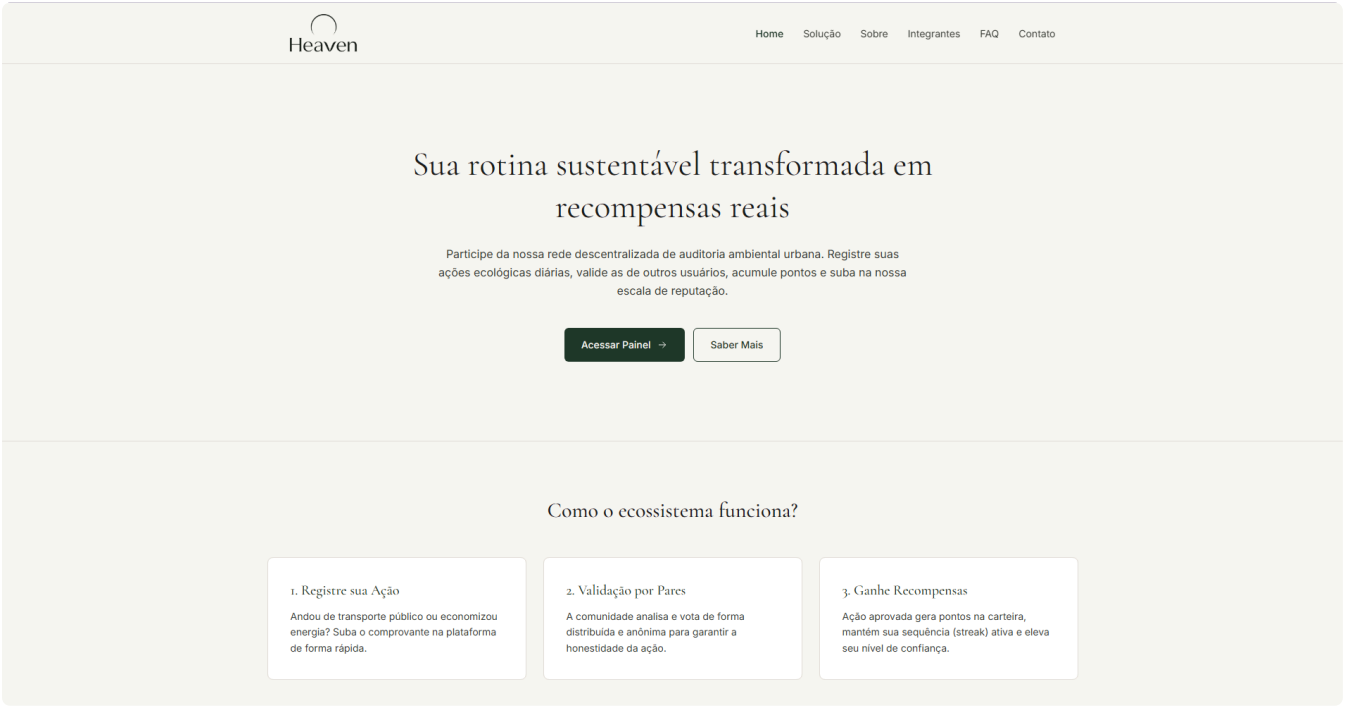
Nas classes de modelo, o construtor vazio, o construtor completo e os getters e setters de cada atributo foram omitidos para manter o diagrama legível. Os métodos das camadas Resource, BO e DAO aparecem no diagrama de camadas.

Fontes dos diagramas no repositório: `docs/classes-camadas.mmd` e `docs/classes-modelo.mmd`.

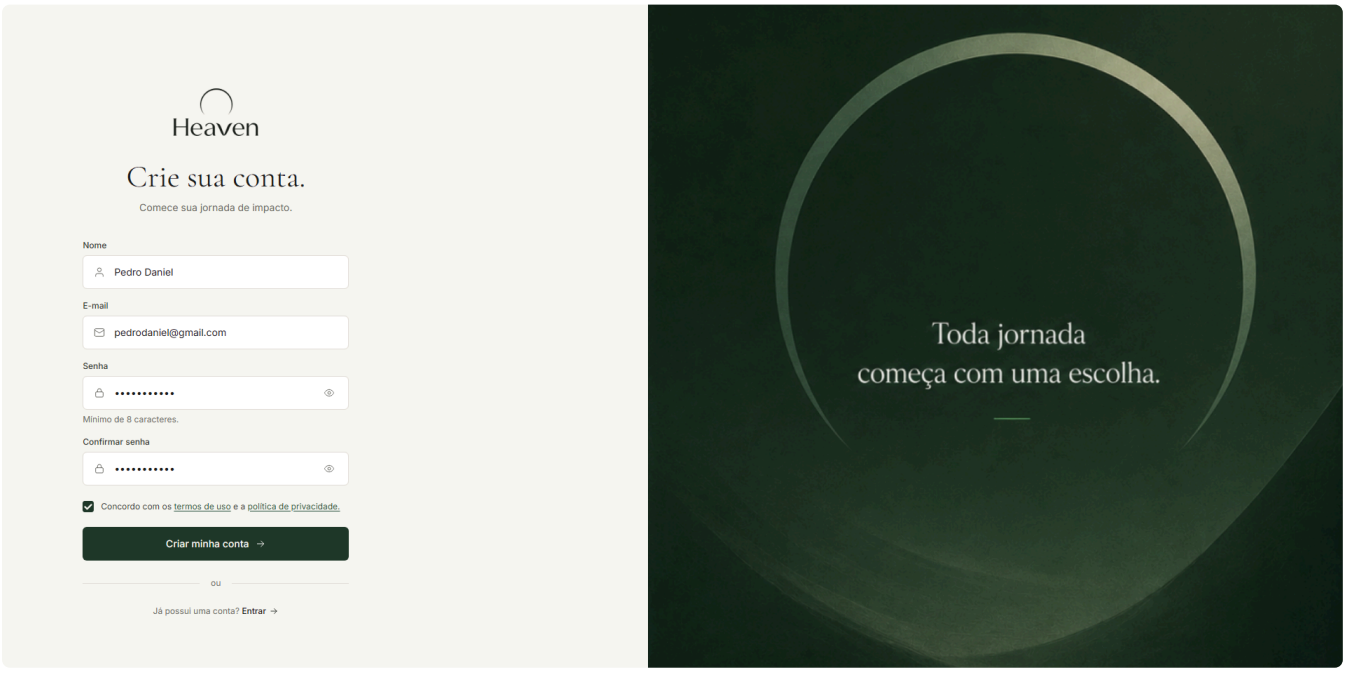
8. Telas da aplicação

Prints do front em produção (Vercel), consumindo a API do Render com dados reais do Oracle.

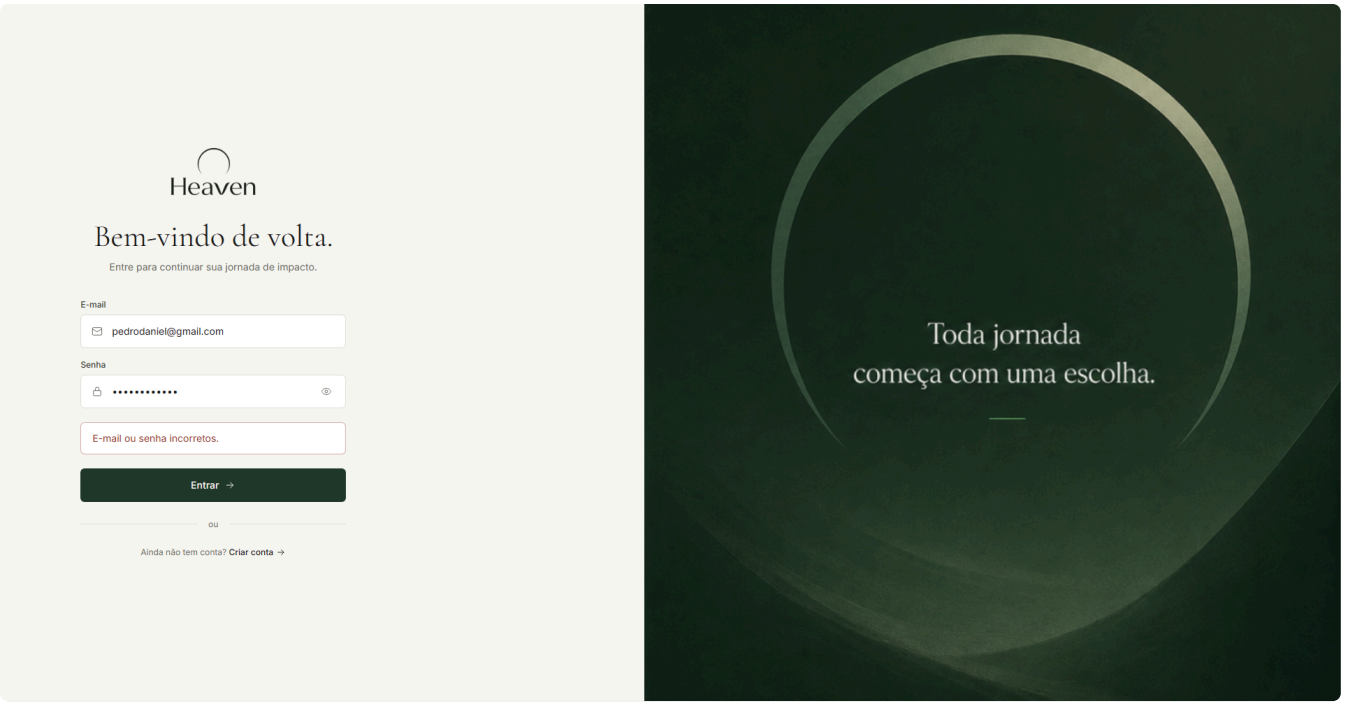
Início



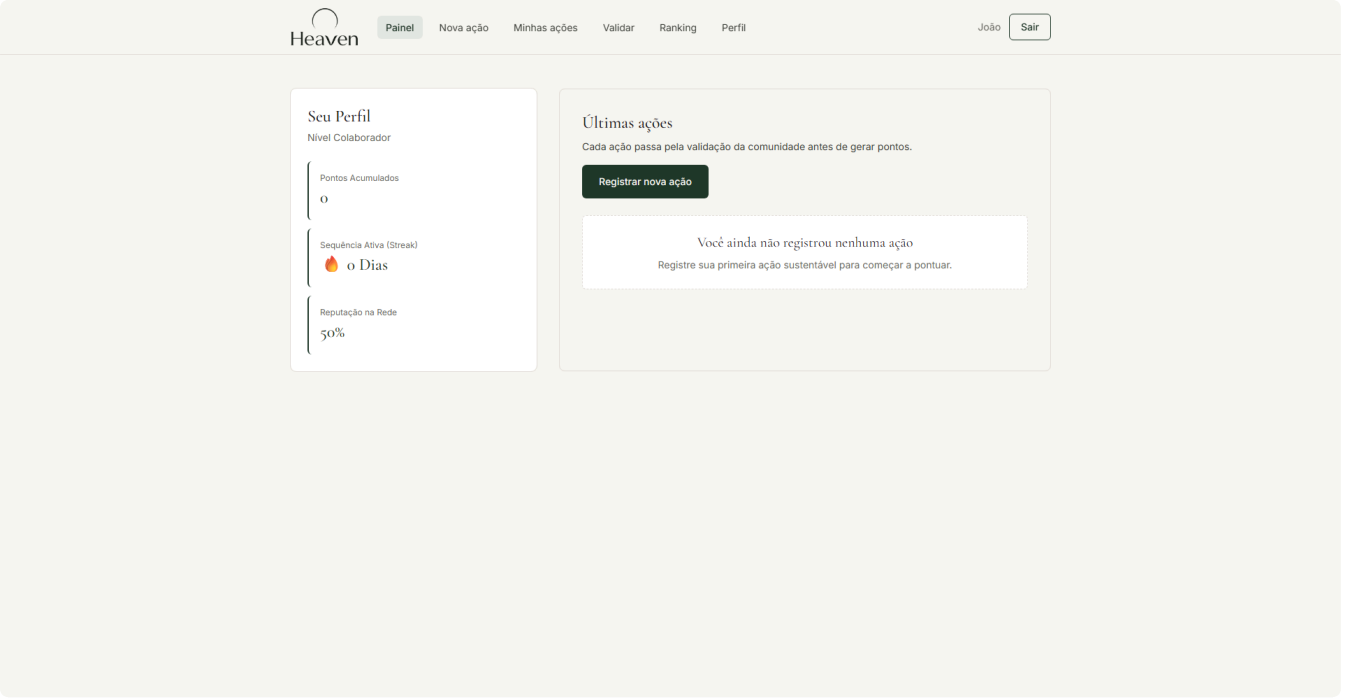
Cadastro — `POST /api/usuarios`



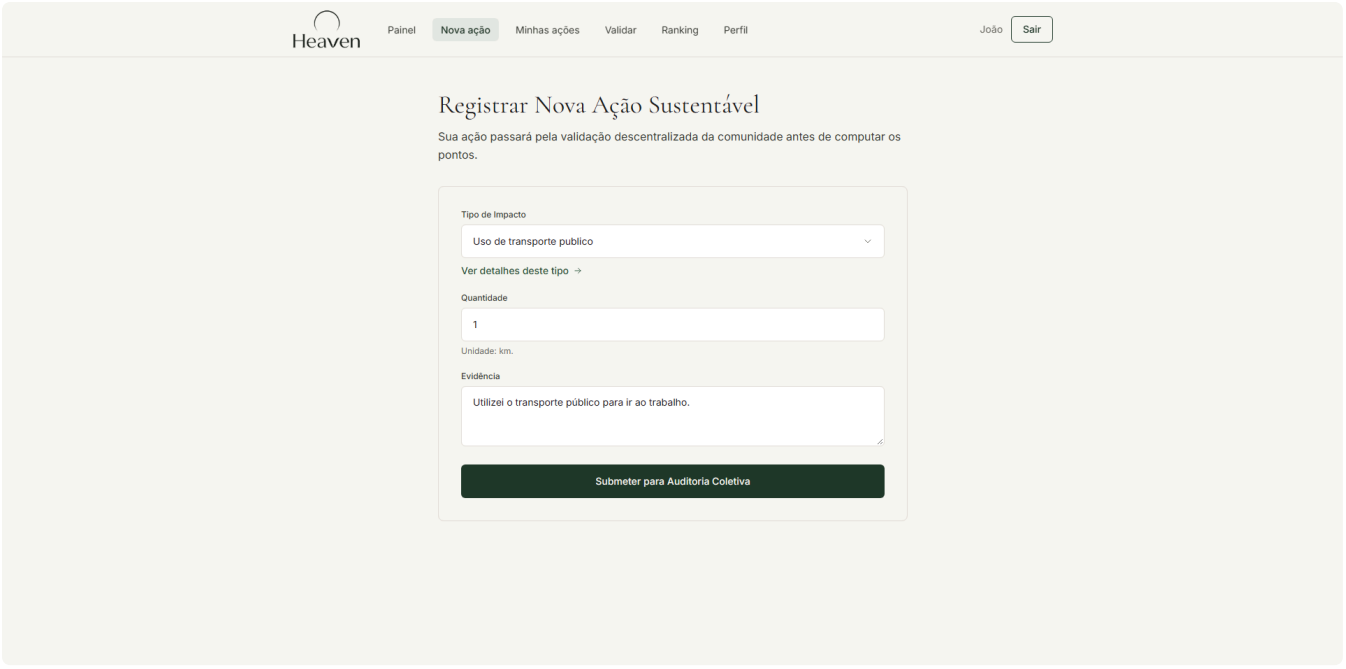
Login com erro — `POST /api/auth/login` devolve 401 e o front mostra a mensagem padronizada



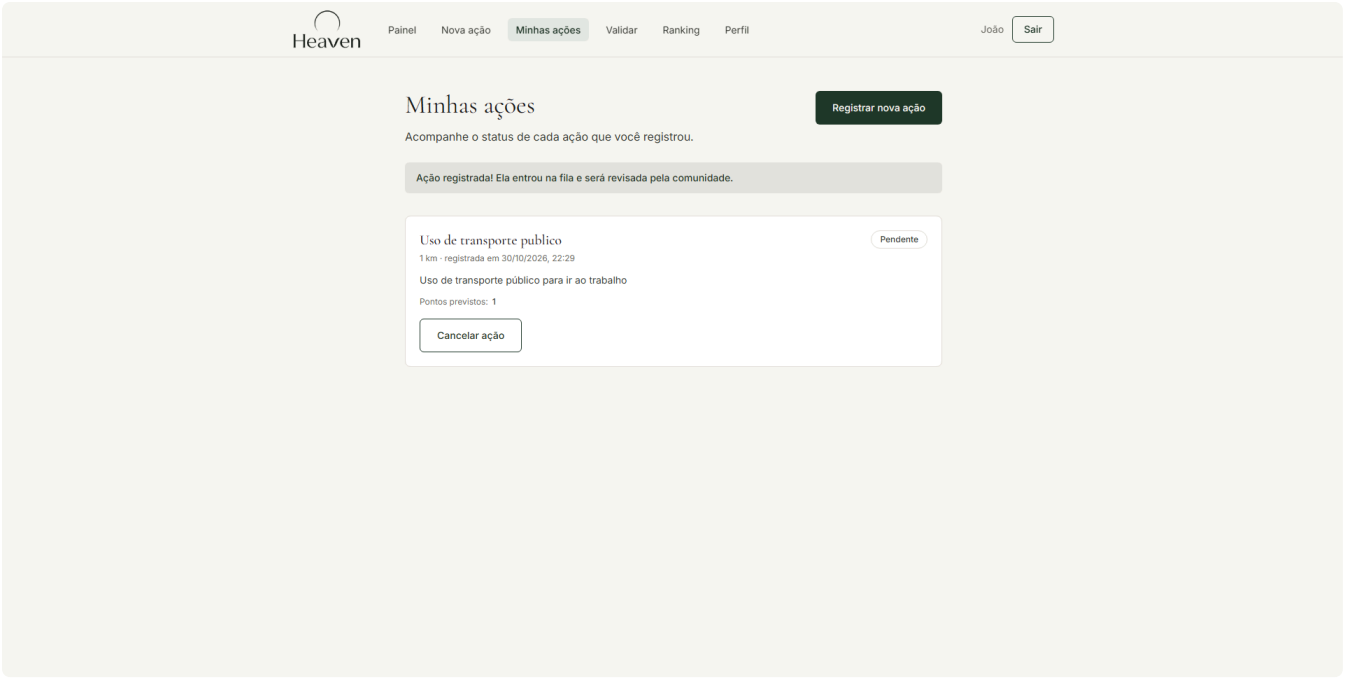
Painel



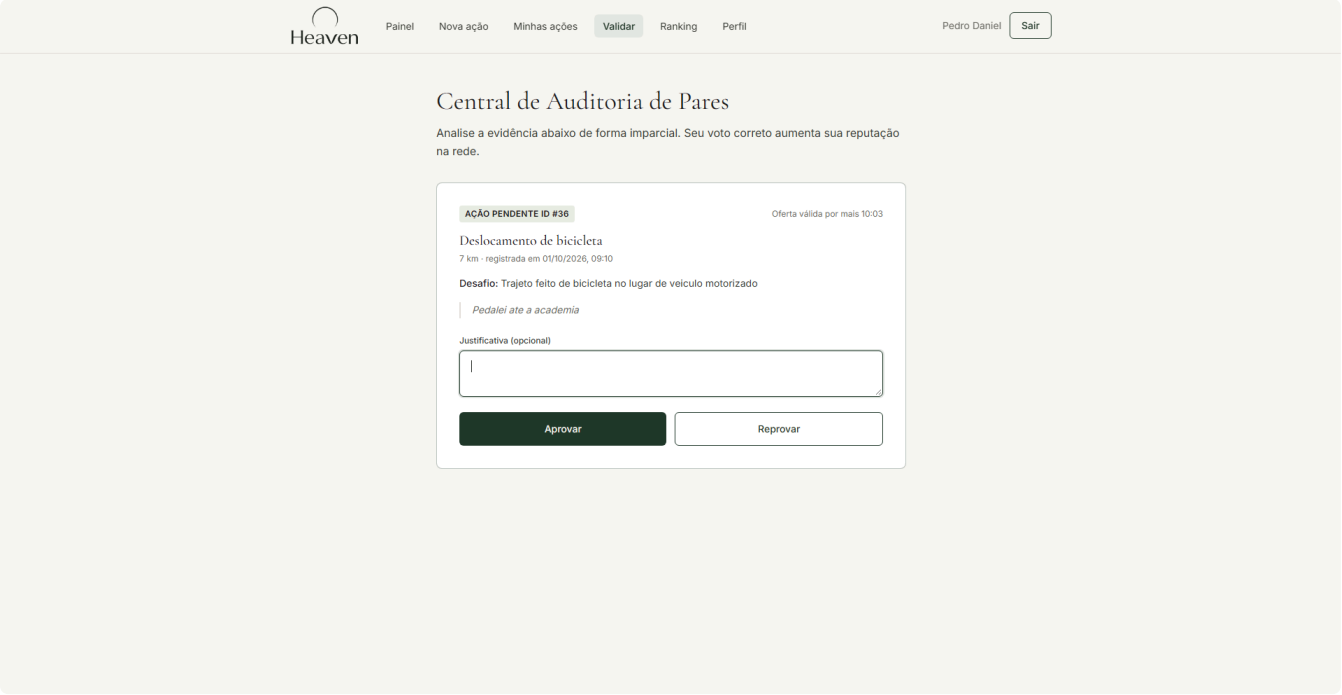
Nova ação — `POST /api/acoes`



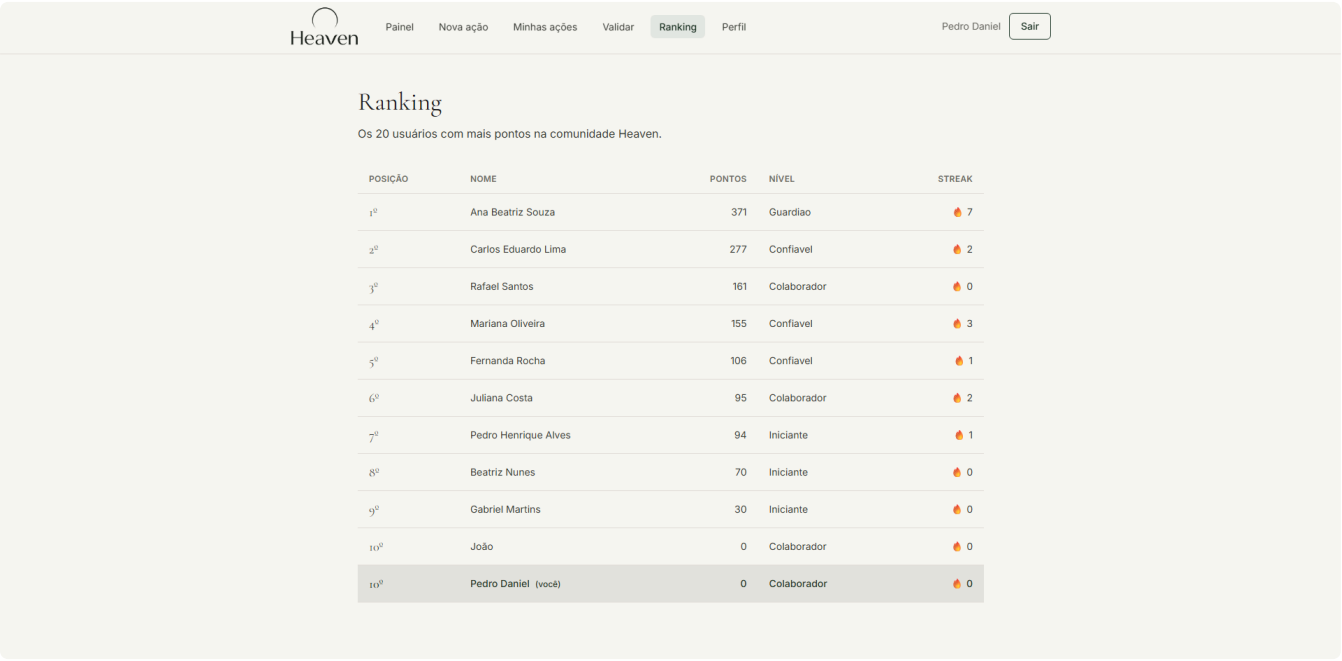
Minhas ações — `GET /api/acoes/minhas`



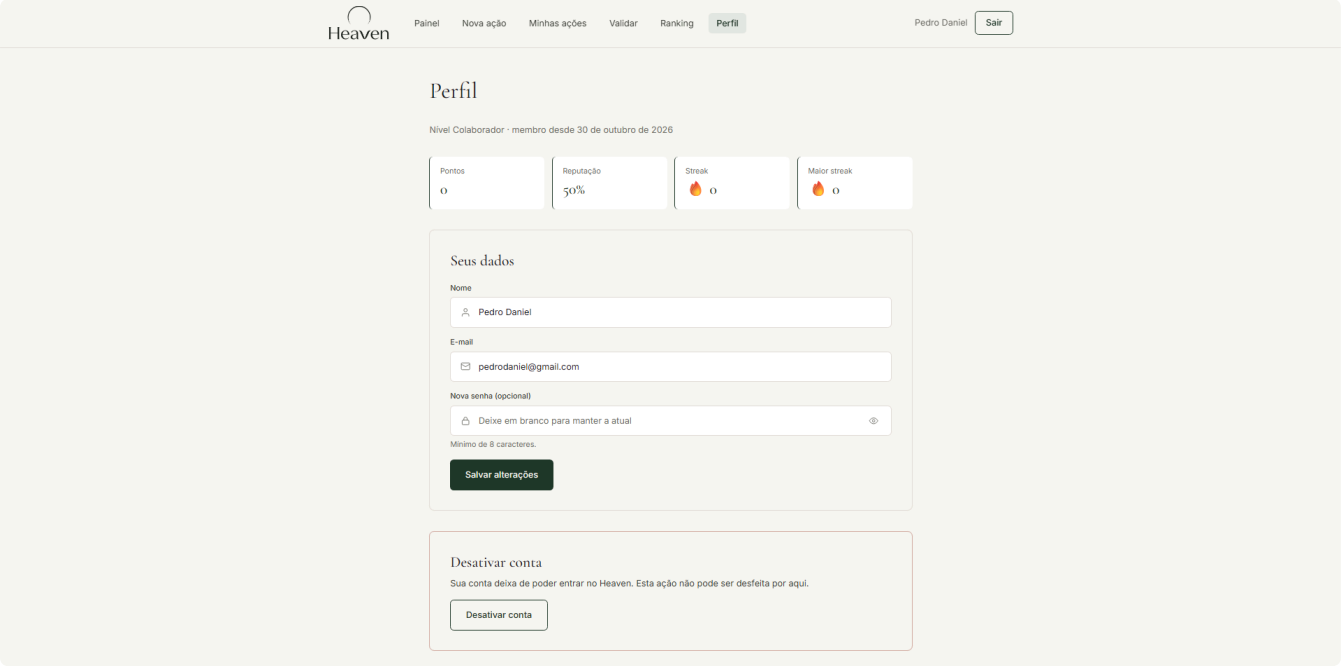
Validar — `POST /api/validacoes/sorteio` e `POST /api/validacoes`, com o tempo restante da oferta



Ranking — `GET /api/ranking`



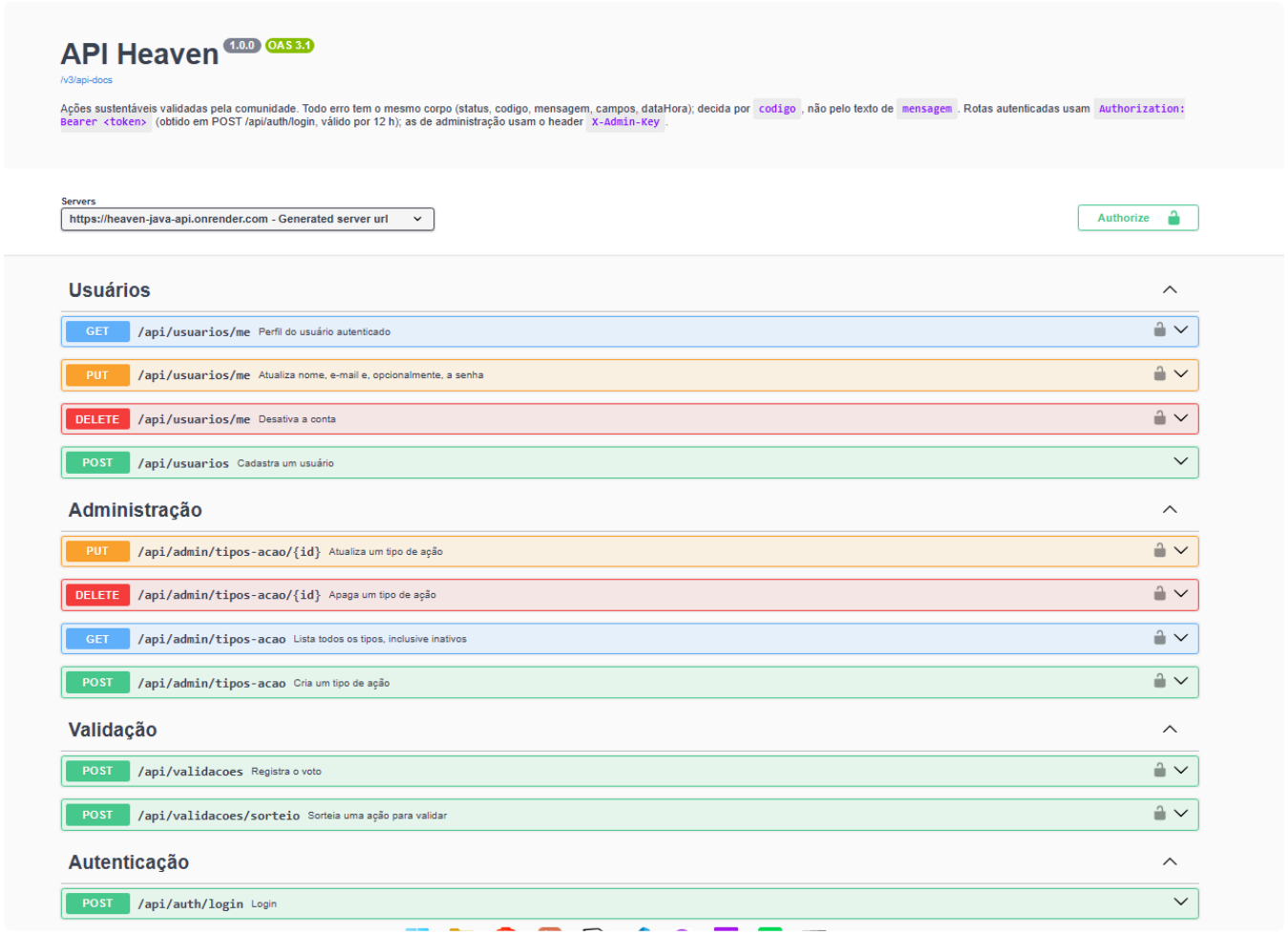
Perfil — `GET`, `PUT` e `DELETE /api/usuarios/me`

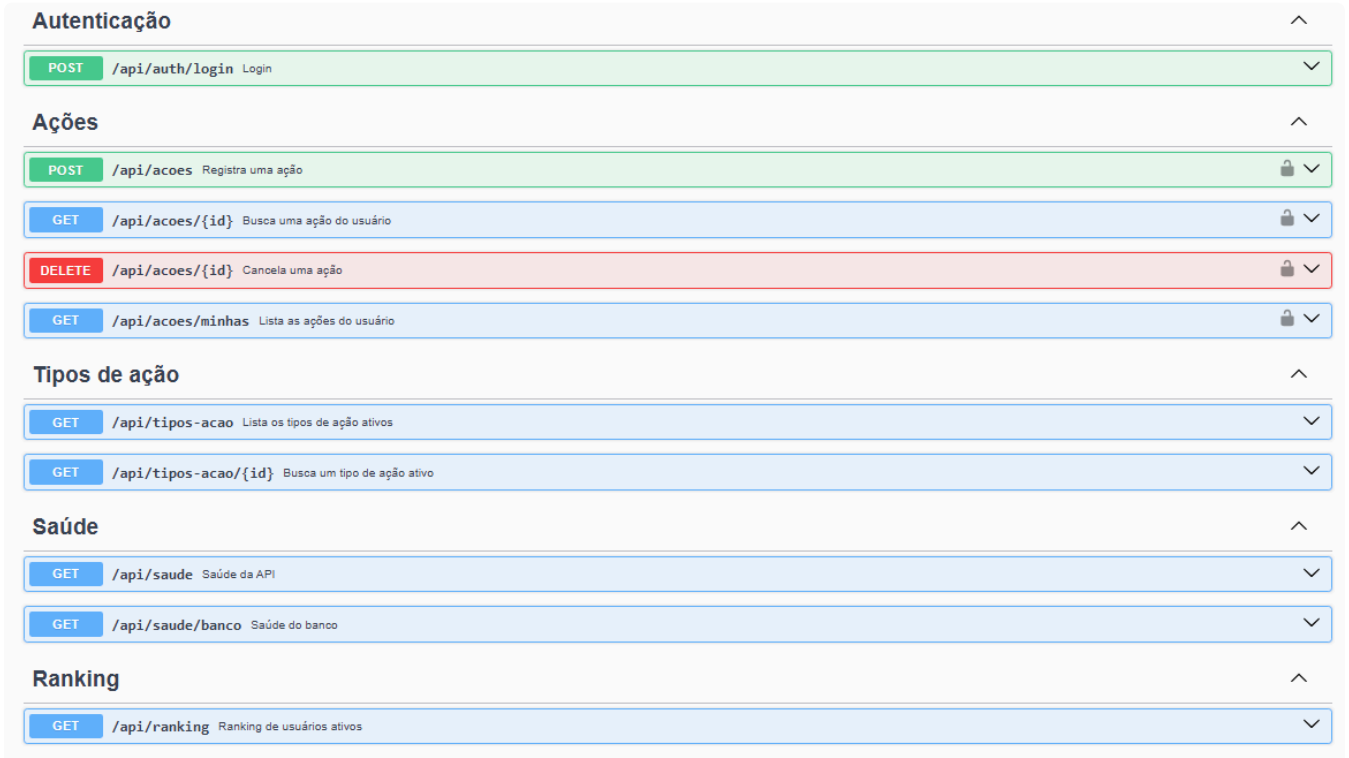


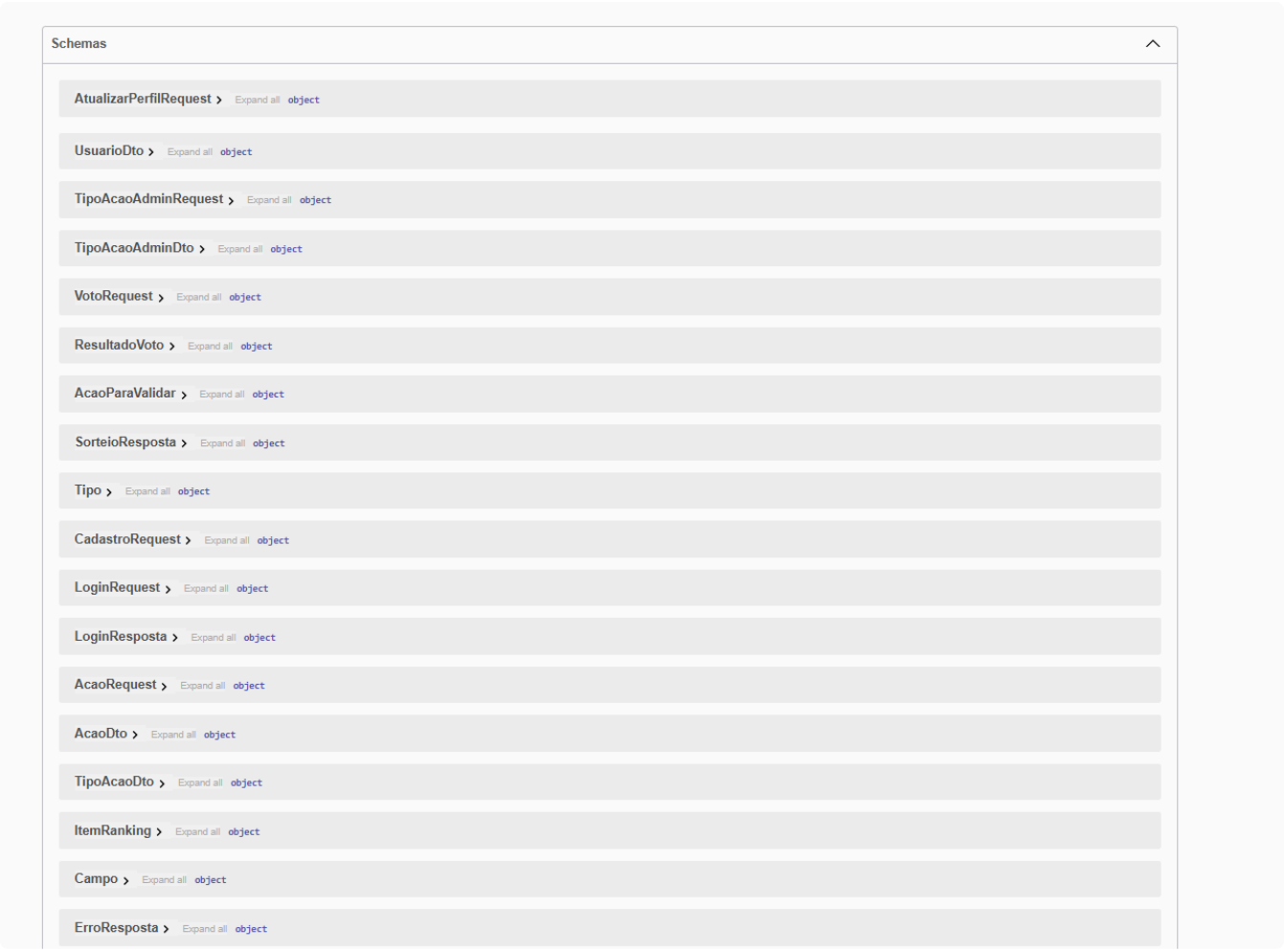
Saúde do banco em produção — `GET /api/saude/banco`



Swagger (documentação interativa) — `/swagger-ui/index.html`, com todos os endpoints agrupados por recurso e os esquemas de entrada e saída.


API Heaven
1.0.0
OAS 3.1
/v3/api-docs
Ações sustentáveis validadas pela comunidade. Todo erro tem o mesmo corpo (status, codigo, mensagem, campos, dataHora); decida por codigo, não pelo texto de mensagem. Rotas autenticadas usam Authorization: Bearer <token> (obtido em POST /api/auth/login, válido por 12 h); as de administração usam o header X-Admin-Key.
Servers
https://heaven-java-api.onrender.com - Generated server url
Authorize
Usuários
GET /api/usuarios/me Perfil do usuário autenticado
PUT /api/usuarios/me Atualiza nome, e-mail e, opcionalmente, a senha
DELETE /api/usuarios/me Desativa a conta
POST /api/usuarios Cadastra um usuário
Administração
PUT /api/admin/tipos-acao/{id} Atualiza um tipo de ação
DELETE /api/admin/tipos-acao/{id} Apaga um tipo de ação
GET /api/admin/tipos-acao Lista todos os tipos, inclusive inativos
POST /api/admin/tipos-acao Cria um tipo de ação
Validação
POST /api/validacoes Registra o voto
POST /api/validacoes/sorteio Sorteia uma ação para validar
Autenticação
POST /api/auth/login Login


Autenticação
POST /api/auth/login Login
Ações
POST /api/acoes Registra uma ação
GET /api/acoes/{id} Busca uma ação do usuário
DELETE /api/acoes/{id} Cancela uma ação
GET /api/acoes/minhas Lista as ações do usuário
Tipos de ação
GET /api/tipos-acao Lista os tipos de ação ativos
GET /api/tipos-acao/{id} Busca um tipo de ação ativo
Saúde
GET /api/saude Saúde da API
GET /api/saude/banco Saúde do banco
Ranking
GET /api/ranking Ranking de usuários ativos



9. Como executar

Requisitos: JDK 17 ou superior e acesso ao Oracle da FIAP (`oracle.fiap.com.br:1521`, SID `orcl`). O Maven vem pelo Wrapper (`mvnw`).

Ferramentas utilizadas: IntelliJ IDEA (IDE do desenvolvimento), JDK 17, Maven Wrapper (Maven 3.9.16), Spring Boot 3.5.16, driver `ojdbc11` 23.6.0.24.10, springdoc-openapi 2.8.13 (Swagger UI) e Oracle SQL Developer para o banco de dados.

Como importar o projeto: no IntelliJ IDEA, use *File > Open* e escolha a pasta do projeto (a que contém o `pom.xml`). O IntelliJ reconhece o Maven e baixa as dependências sozinho. Em *File > Project Structure*, confirme o JDK 17 como SDK do projeto. Em *Run > Edit Configurations*, defina as variáveis de ambiente da tabela abaixo na configuração da aplicação Spring Boot e execute. Quem preferir o terminal pode seguir os passos logo depois da tabela.

Variáveis de ambiente (nenhuma credencial fica no repositório):

Variável	Uso
`SPRING_DATASOURCE_URL`	URL JDBC do Oracle
`SPRING_DATASOURCE_USERNAME`	usuário do banco
`SPRING_DATASOURCE_PASSWORD`	senha do banco
`HEAVEN_TOKEN_SECRET`	segredo do token (mínimo 32 caracteres)
`HEAVEN_ADMIN_KEY`	chave dos endpoints de administração (mínimo 24 caracteres)
`HEAVEN_CORS_ORIGINS`	origens permitidas pelo CORS
`PORT`	porta HTTP (usada pelo Render)

Passos

1. Clonar o repositório e definir as variáveis acima.
2. Gerar o pacote: `./mvnw clean package`.
3. Executar: `java -jar target/heaven.jar`.
4. Conferir: `GET /api/saude` e `GET /api/saude/banco` devem responder 200.

O repositório também traz `Dockerfile`, usado no deploy do Render. A criação das tabelas é feita pelo script `database/Heaven-original.sql` da disciplina de Banco de Dados.

Deploy: a API roda no Render (plano gratuito, que hiberna após 15 minutos sem uso; a primeira chamada pode levar cerca de 1 minuto). O front roda na Vercel e aponta para a API pela variável `VITE_API_URL`.